12.2 The Golden Rules

In his book on interface design, Theo Mandel [Man97] coins three *golden rules:*

1. Place the user in control.
2. Reduce the user's memory load.
3. Make the interface consistent.

These golden rules actually form the basis for a set of user interface design principles that guide this important aspect of software design.

12.2.1 Place the User in Control

During a requirements gathering session for a major new information system, a key user was asked about the attributes of the window-oriented graphical interface.

"What I really would like," said the user solemnly, "is a system that reads my mind. It knows what I want to do before I need to do it and makes it very easy for me to get it done. That's all, just that."

Your first reaction might be to shake your head and smile, but pause for a moment. There was absolutely nothing wrong with the user's request. She wanted a system that reacted to her needs and helped her get things done. She wanted to control the computer, not have the computer control her.

Most interface constraints and restrictions that are imposed by a designer are intended to simplify the mode of interaction. But for whom?

As a designer, you may be tempted to introduce constraints and limitations to simplify the implementation of the interface. The result may be an interface that is easy to build, but frustrating to use. Mandel [Man97] defines a number of design principles that allow the user to maintain control:

Define interaction modes in a way that does not force a user into unnecessary or undesired actions. An interaction mode is the current state of the interface. For example, if *autocorrect* is selected in a text messaging app menu, the software performs autocorrect continually. There is no reason to force the user to remain in autocorrect mode. The user should be able to enter and exit the mode with little or no effort.

Provide for flexible interaction. Because different users have different interaction preferences, choices should be provided. For example, software might allow a user to interact via keyboard commands, mouse movement, a digitizer pen, a multitouch screen, or voice recognition commands. But every action is not amenable to every interaction mechanism. Consider, for example, the difficulty of using a keyboard command (or voice input) to draw a complex shape.

Allow user interaction to be interruptible and undoable. Even when involved in a sequence of actions, the user should be able to interrupt the sequence to do something else (without losing the work that had been done). The user should also be able to "undo" any action or any linear sequence of actions.

Streamline interaction as skill levels advance, and allow the interaction to be customized. Users often find that they perform the same sequence of interactions repeatedly. It is worthwhile to design a "macro" mechanism that enables an advanced user to customize the interface to facilitate interaction.

Hide technical internals from the casual user. The user interface should move the user into the virtual world of the application. The user should not be aware of the operating system, file management functions, or other arcane computing technology.

Design for direct interaction with objects that appear on the screen. The user feels a sense of control when able to manipulate the objects that are necessary to perform a task in a manner similar to what would occur if the object were a physical thing. For example, an application interface that allows a user to drag a document into the "trash" is an implementation of direct manipulation.

12.2.2 Reduce the User's Memory Load

A well-designed user interface does not tax a user's memory because the more a user has to remember, the more error-prone the interaction will be. Whenever possible, the system should "remember" pertinent information and assist the user with an interaction scenario that assists recall. Mandel [Man97] defines design principles that enable an interface to reduce the user's memory load:

Reduce demand on short-term memory. When users are involved in complex tasks, the demand on short-term memory can be significant. The interface should be designed to reduce the requirement to remember past actions, inputs, and results. This can be accomplished by providing visual cues that enable a user to recognize past actions, rather than having to recall them.

Establish meaningful defaults. The initial set of defaults should make sense for the average user, but a user should be able to specify individual preferences. However, a "reset" option should be available, enabling the redefinition of original default values.

Define shortcuts that are intuitive. When mnemonics are used to accomplish a system function (e.g., ctrl-C to invoke the *copy* function), the mnemonic should be tied to the action in a way that is easy to remember (e.g., first letter of the task to be invoked).

The visual layout of the interface should be based on a real-world metaphor. For example, a bill payment system should use a checkbook and check register metaphor to guide the user through the bill paying process. A room layout application should allow users to drag furniture from a visual catalog and arrange it on the screen using a touch interface. This enables the user to rely on well-understood visual cues, rather than memorizing an arcane interaction sequence.

Disclose information in a progressive fashion. The interface should be organized hierarchically. That is, information about a task, an object, or some behavior should be presented first at a high level of abstraction. More detail should be presented after the user indicates interest.

SAFEHOME



Violating a User Interface Golden Rule

The scene: Vinod's cubicle, as user interface design begins.

The players: Vinod and Jamie, members of the *SafeHome* software engineering team.

The conversation:

Jamie: I've been thinking about the surveillance function interface.

Vinod (smiling): Thinking is good.

Jamie: I think maybe we can simplify matters some.

Vinod: Meaning?

Jamie: Well, what if we eliminate the floor plan entirely? It's flashy, but it's going to take serious development effort. Instead, we just ask the user to specify the camera he wants to see and then display the video in a video window.

Vinod: How does the homeowner remember how many cameras are set up and where they are?

Jamie (mildly irritated): He's the homeowner; he should know.

Vinod: But what if he doesn't?

Jamie: He should.

Vinod: That's not the point . . . what if he forgets?

Jamie: Uh, we could provide a list of operational cameras and their locations.

Vinod: That's possible, but why should he have to ask for a list?

Jamie: Okay, we provide the list whether he asks or not.

Vinod: Better. At least he doesn't have to remember stuff that we can give him.

Jamie (thinking for a moment): But you like the floor plan, don't you?

Vinod: Uh-huh. Especially since we are creating the room layout application in a related product.

Jamie: Which one will marketing like, do you think?

Vinod: You're kidding, right?

Jamie: No.

Vinod: Duh . . . the one with the flash . . . they love sexy product features . . . they're not interested in which is easier to build.

Jamie (sighing): Okay, maybe I'll create paper prototypes of both.

Vinod: Good idea . . . then we let the customer decide.

12.2.3 Make the Interface Consistent

The interface should present and acquire information in a consistent fashion. This implies that (1) all visual information is organized according to design rules that are maintained throughout all screen displays, (2) input mechanisms are constrained to a limited set that is used consistently throughout the application, and (3) mechanisms for navigating from task to task are consistently defined and implemented. Mandel [Man97] defines a set of design principles that help make the interface consistent:

Allow the user to put the current task into a meaningful context. Many interfaces implement complex layers of interactions with dozens of screen

images. It is important to provide indicators (e.g., window titles, graphical icons, consistent color coding) that enable the user to know the context of the work at hand. In addition, the user should be able to determine where he has come from and what alternatives exist for a transition to a new task.

Maintain consistency across a complete product line. A family of applications (i.e., a product line) should implement the same design rules so that consistency is maintained for all interaction.

If past interactive models have created user expectations, do not make changes unless there is a compelling reason to do so. Once a particular interactive sequence has become a de facto standard (e.g., the use of alt-S to save a file), the user expects this in every application encountered. A change (e.g., using alt-S to invoke scaling) will cause confusion.

The interface design principles discussed in this and the preceding sections provide you with basic guidance. In the sections that follow, you'll learn about the user experience design process itself.

12.3 User Interface Analysis and Design

Although UX design work is not only about the user interface, design of the user interface is a good place to start understanding the UX process. The overall process for analyzing and designing a user interface begins with the creation of different models of system function (as perceived from the outside). You begin by delineating the human- and computer-oriented tasks that are required to achieve system function and then considering the design issues that apply to all interface designs. Tools are used to prototype and ultimately implement the design model, and the result is evaluated by end users for quality.

12.3.1 Interface Analysis and Design Models

Four different models come into play when a user interface is to be analyzed and designed. A human engineer (or the software engineer) establishes a *user model,* the software engineer creates a *design model,* the end user develops a mental image that is often called the user's *mental model* or the *system perception,* and the implementers of the system create an *implementation model*. Unfortunately, each of these models may differ significantly. Your role, as an interface designer, is to reconcile these differences and derive a consistent representation of the interface.

The user model establishes the profile of end users of the system. To build an effective user interface, "all design should begin with an understanding of the intended users, including profiles of their age, gender, physical abilities, education, cultural or ethnic background, motivation, goals and personality" [Shn16]. In addition, users can be categorized as novices; knowledgeable, intermittent users; or knowledgeable frequent users. Many UX designers like to build user profiles or personas (Section 12.4.2) as way of capturing what is known about each class of users.

The user's *mental model* (system perception) is the image of the system that end users carry in their heads. For example, if the user of a mobile app that rates restaurants were asked to describe its operation, the system perception would guide the

response. The accuracy of the description will depend on the user's profile (e.g., novices would provide a sketchy response at best) and overall familiarity with software in the application domain. A user who understands restaurant rating apps fully but has worked with the specific app only a few times might actually be able to provide a more complete description of its function than the novice who has spent days trying to apply the app effectively.

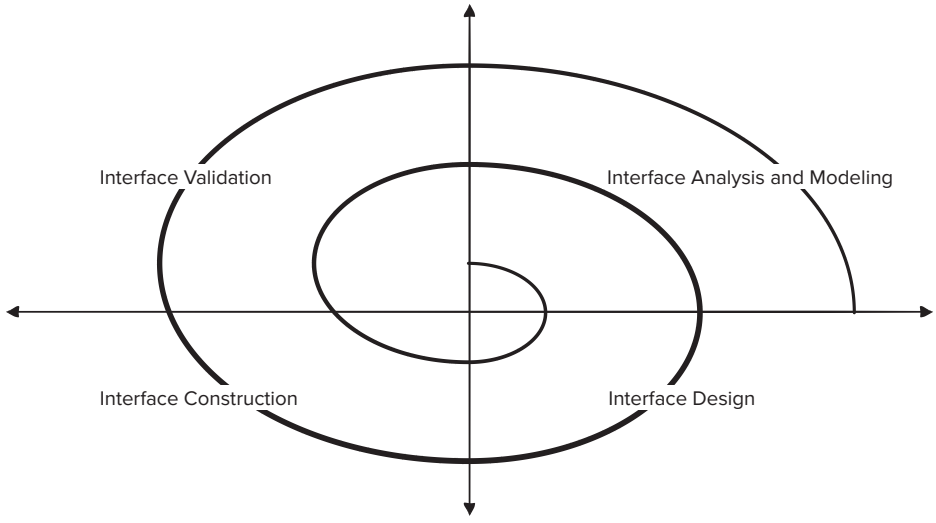
The *implementation model* combines the outward manifestation of the computer-based system (the look and feel of the interface), coupled with all supporting information (books, manuals, videotapes, help files) that describes interface syntax and semantics. When the implementation model and the user's mental model are coincident, users generally feel comfortable with the software and use it effectively. To accomplish this "melding" of the models, the design model must have been developed to accommodate the information contained in the user model, and the implementation model must accurately reflect syntactic and semantic information about the interface.

12.3.2 The Process

The analysis and design process for user interfaces is iterative and can be represented using a process model similar to the one discussed in Chapter 4. Referring to Figure 12.2, the user interface analysis and design process begins at the interior of the spiral and encompasses four distinct framework activities [Man97]: (1) interface analysis and modeling, (2) interface design, (3) interface construction, and (4) interface validation. The spiral shown in Figure 12.2 implies that each of these tasks will occur more than once, with each pass around the spiral representing additional elaboration of requirements and the resultant design. In most cases, the construction activity involves prototyping—the only practical way to validate what has been designed.

FIGURE 12.2

The user interface design process



Interface analysis focuses on the profile of the users who will interact with the system. Skill level, business understanding, and general receptiveness to the new system are recorded; and different user categories are defined. For each user category, requirements are elicited. In essence, you work to understand the system perception (Section 12.4.2) for each class of users.

Once general requirements have been defined, a more detailed *task analysis* is conducted. Those tasks that the user performs to accomplish the goals of the system are identified, described, and elaborated (over a number of iterative passes through the spiral). Task analysis is discussed in more detail in Section 12.4.3. Finally, analysis of the user environment focuses on the characteristics of the physical work environment (e.g., location, lighting, position constraints).

The information gathered as part of the analysis action is used to create an analysis model for the interface. Using this model as a basis, the design activity commences.

The goal of *interface design* is to define a set of interface objects and actions (and their screen representations) that enable a user to perform all defined tasks in a manner that meets every usability goal defined for the system. Interface design is discussed in more detail in Section 12.5.

Interface construction normally begins with the creation of a prototype that enables usage scenarios to be evaluated. As the iterative design process continues, a user interface tool kit may be used to complete the construction of the interface.

Interface validation focuses on (1) the ability of the interface to implement every user task correctly, to accommodate all task variations, and to achieve all general user requirements; (2) the degree to which the interface is easy to use and easy to learn, and (3) the user's acceptance of the interface as a useful tool in her work.

As we have already noted, the activities described in this section occur iteratively. Therefore, there is no need to attempt to specify every detail (for the analysis or design model) on the first pass. Subsequent passes through the process elaborate task detail, design information, and the operational features of the interface.

12.4 USER EXPERIENCE ANALYSIS[2]

A key tenet of all software engineering process models is this: *Understand the problem before you attempt to design a solution.* In the case of user experience design, understanding the problem means understanding (1) the people (end users) who will interact with the system through the interface, (2) the tasks that end users must perform to do their work, (3) the content that is presented as part of the interface, and (4) the environment in which these tasks will be conducted. In the sections that follow, we examine each of these elements of UX analysis with the intent of establishing a solid foundation for the interface design tasks that follow.

2 It is reasonable to argue that this section should be placed in Chapter 8, since requirements analysis issues are discussed there. It has been positioned here because user experience analysis and design are intimately connected to one another, and the boundary between the two is often fuzzy.